



Parul University

FACULTY OF ENGINEERING AND TECHNOLOGY
BACHELOR OF TECHNOLOGY

DEEP LEARNING WITH NLP LABORATORY
(303105480)

7TH SEMESTER

ARTIFICIAL INTELLIGENCE & DATA SCIENCE (A.I & D.S) DEPARTMENT

LABORATORY MANUAL

CERTIFICATE

This is to certify that

Ms. Gopi Kota with Enrollment No. 2303031247002 has successfully completed her laboratory experiments in Deep Learning with NLP (303105480) from the department of Artificial Intelligence & Data Science (A.I & D.S) during the academic year 2025-2026.



Date of Submission

Staff In charge

Head of Department.....

TABLE OF CONTENTS

SR.NO	Experiment Title	Page No		Date of Performance	Date of Submission	Marks	Sign
		From	To				
1	Implementation of preprocessing of Text with NLTK (Tokenization, Stemming, Lemmatization) and removal of stop words in NLP.						
2	Implementation to Convert the text to word count vectors with ScikitLearn (CountVectorizer).						
3	Implementation to Convert the text to word frequency vectors with ScikitLearn (TfidfVectorizer).						
4	Implementation to Convert the text to unique integers with ScikitLearn (HashingVectorizer).						
5	Use the Keras deep learning library and split words with (text_to_word_sequence)						
6	Use the Keras deep learning library and write a code for encoding with (one_hot).						
7	Use the Keras deep learning library and write a code for Hash Encoding with (hashing_trick).						
8	Use the Keras deep learning library give a demo of Tokenizer API						
9	Perform an experiment to do sentiment analysis on any of the dataset (like twitter tweets, movie review etc.).						
10	Perform an experiment using Gensim python library for Word2Vec Embedding.						

PRACTICAL: 01

Aim: Implementation of preprocessing of Text with NLTK (Tokenization, Stemming, Lemmatization) and removal of stop words in NLP.

Objective: The objective of this practical is to understand and apply basic text preprocessing techniques using NLTK that prepare raw textual data for NLP tasks such as classification, sentiment analysis, or information retrieval.

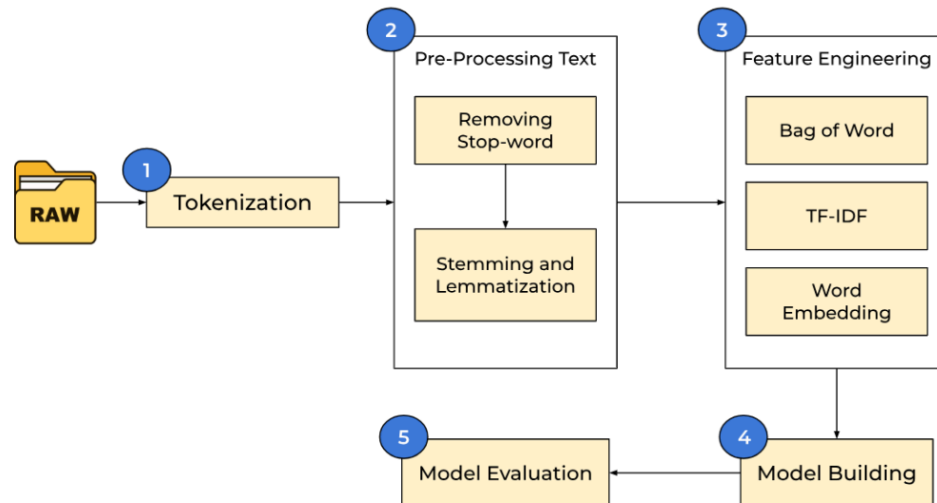
Theory:

- **Text preprocessing** is an essential step in NLP to convert raw text into a clean and analyzable format. Common preprocessing tasks include:
 - **Tokenization:** Splits text into individual words or tokens.
 - **Stemming:** Reduces words to their root form by chopping off suffixes (e.g., "playing" → "play").
 - **Lemmatization:** Reduces words to their dictionary form (lemma) considering grammar (e.g., "better" → "good").
 - **Stop Word Removal:** Eliminates common words (e.g., "is", "the") that do not contribute to meaning in many NLP tasks.
- These techniques help reduce dimensionality and improve the performance of NLP models.

About NLTK:

- NLTK (Natural Language Toolkit) is a powerful Python library used for working with human language data (text). It provides easy-to-use tools for tokenization, stemming, lemmatization, stop word removal, parsing, and more. NLTK is widely used in research and education for performing natural language processing tasks and building NLP models.

NLTK :



Explanation of the Text Processing Pipeline:

1. Tokenization:

- Splits raw text into words or tokens.
- Example: "The cat sat" → ["The", "cat", "sat"]

2. Pre-Processing Text:

- Removing Stop-words: Eliminates common words (e.g., "the", "is") that carry less meaning.
- Stemming & Lemmatization: Reduces words to their base or root form.
 - Example: "running" → "run"

3. Feature Engineering:

- Converts processed text into numerical form for model input:
 - Bag of Words (BoW): Counts word occurrences.
 - TF-IDF (Term Frequency - Inverse Document Frequency): Measures word importance.
 - Word Embedding: Represents words in dense vector form (semantic meaning).

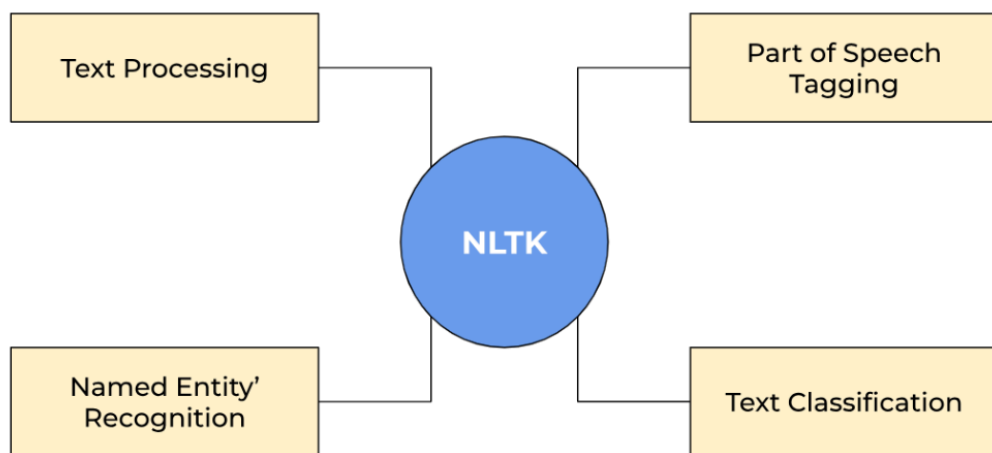
4. Model Building:

- Uses the engineered features to train machine learning or deep learning models.

5. Model Evaluation:

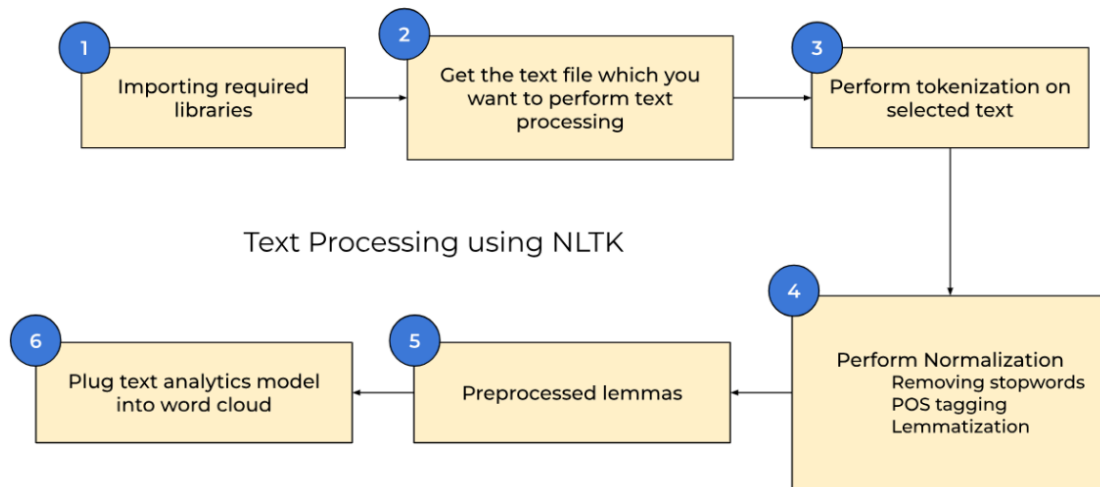
- Assesses model performance using metrics like accuracy, precision, recall, etc.

Natural Language Processing with NLTK – Key Capabilities:



- **Text Processing** - Basic tasks like tokenization, stop word removal, stemming, and lemmatization.
- **Part of Speech (POS) Tagging** - Identifies grammatical roles of words (e.g., noun, verb, adjective).
- **Named Entity Recognition (NER)** - Detects entities like names, locations, dates in text (e.g., "India", "Google").
- **Text Classification** - Categorizes text into predefined labels (e.g., spam vs. not spam, sentiment analysis).

Text Processing using NLTK – Steps Explained:



1. Importing Required Libraries

- Load necessary Python packages like `nltk` for NLP tasks.

2. Load Text File

- Open and read the input text file that you want to process.

3. Tokenization

- Break the text into smaller units like words or sentences using `word_tokenize()` or `sent_tokenize()`.

4. Perform Normalization

- Clean and standardize the text:
 - **Remove Stopwords** (e.g., "the", "is")
 - **POS Tagging** (assign parts of speech to each word)
 - **Lemmatization** (convert words to their base forms)

5. Preprocessed Lemmas

- The output is a clean list of lemmatized, tagged, and filtered words ready for analysis.

6. Plug into WordCloud for Analytics Model

- Use the final cleaned data for visualizations like WordCloud or feed it into a text analytics or ML model.

Code:

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer

# Download required resources
# nltk.download('punkt')
# nltk.download('stopwords')
# nltk.download('wordnet')
# or use nltk.download('all')

# Sample input
text = "My Name is Gopi Kota This one is NLP first Practical."

# Tokenization
tokens = word_tokenize(text)

# Stopword Removal
stop_words = set(stopwords.words('english'))
filtered_tokens = [word for word in tokens if word.lower() not in stop_words]

# Stemming
stemmer = PorterStemmer()
stemmed_tokens = [stemmer.stem(word) for word in filtered_tokens]

# Lemmatization
lemmatizer = WordNetLemmatizer()
lemmatized_tokens = [lemmatizer.lemmatize(word) for word in filtered_tokens]

# Output
print("Original Text:", text)
print("Tokens:", tokens)
print("After Stopword Removal:", filtered_tokens)
print("After Stemming:", stemmed_tokens)
print("After Lemmatization:", lemmatized_tokens)
```

Input:

- **My Name is Gopi Kota This one is NLP first Practical."**

Output:

```
⇒ Original Text: My Name is Gopi Kota This one is NLP first Practical.  
Tokens: ['My', 'Name', 'is', 'Gopi', 'Kota', 'This', 'one', 'is', 'NLP', 'first', 'Practical', '.']  
After Stopword Removal: ['Name', 'Gopi', 'Kota', 'one', 'NLP', 'first', 'Practical', '.']  
After Stemming: ['name', 'gopi', 'kota', 'one', 'nlp', 'first', 'practic', '.']  
After Lemmatization: ['Name', 'Gopi', 'Kota', 'one', 'NLP', 'first', 'Practical', '.']
```

Conclusion:

- In this practical, we successfully implemented basic text preprocessing techniques using the NLTK library. The steps included **tokenization**, **stopword removal**, **stemming**, and **lemmatization**. These techniques are essential for preparing raw text data for further Natural Language Processing tasks, such as classification, sentiment analysis, or language modeling.
- By applying these preprocessing steps, we reduce noise, normalize the data, and extract meaningful information from text, making it more suitable for machine learning models.

PRACTICAL: 02

Aim: Implementation to Convert the text to word count vectors with ScikitLearn (CountVectorizer).

Objective: To understand how to represent textual data in a structured numerical format using the Bag-of-Words model, which helps transform text into fixed-size feature vectors suitable for machine learning algorithms.

Theory:

The **Bag-of-Words (BoW)** model is a simple technique to convert text documents into numerical vectors. It:

- Ignore grammar and word order.
- Focuses on word frequency.
- Represents each document as a vector of word counts.

CountVectorizer is a tool in `sklearn.feature_extraction.text` that:

- Tokenizes the text.
- Builds a vocabulary of known words.
- Encodes new documents using this vocabulary into word count vectors.

This format is commonly used for classification, clustering, and NLP tasks.

Code:

```
#P2
# Install scikit-learn if not already installed
!pip install scikit-learn --quiet

from sklearn.feature_extraction.text import CountVectorizer
```

```
# Sample corpus (text data)
corpus = [
    "Data science is an exciting field",
    "Python is widely used in data analysis",
    "Machine learning plays a key role in data science"
]

# Initialize CountVectorizer
vectorizer = CountVectorizer()

# Fit and transform the corpus
X = vectorizer.fit_transform(corpus)

# Convert result to array and get feature names
print("Vocabulary:", vectorizer.get_feature_names_out())
print("\nWord Count Vectors:\n", X.toarray())
```

Input:

```
corpus = [
    "Data science is an exciting field",
    "Python is widely used in data analysis",
    "Machine learning plays a key role in data science"
]
```

Output:

```
⇒ Vocabulary (Features):
['an' 'analysis' 'data' 'exciting' 'field' 'in' 'is' 'key' 'learning'
 'machine' 'plays' 'python' 'role' 'science' 'used' 'widely']

Word Count Vectors (Matrix):
[[1 0 1 1 1 0 1 0 0 0 0 0 0 1 0 0]
 [0 1 1 0 0 1 1 0 0 0 0 1 0 0 1 1]
 [0 0 1 0 0 1 0 1 1 1 1 0 1 1 0 0]]
```

Conclusion:

- In this practical, we used CountVectorizer from Scikit-learn to convert text documents into numerical word count vectors. This technique allows raw text to be transformed into a format that can be used as input to machine learning models. It is a foundational step in building text classification, sentiment analysis, or information retrieval systems.

PRACTICAL: 03

Aim: Implementation to Convert the text to word frequency vectors with ScikitLearn (TfidfVectorizer).

Objective: To understand how TF-IDF (Term Frequency–Inverse Document Frequency) represents text data based on the importance of words across multiple documents, and convert it into numeric feature vectors for machine learning applications.

Theory:

TF-IDF is a feature extraction technique that reflects how important a word is to a document in a collection.

It combines two metrics:

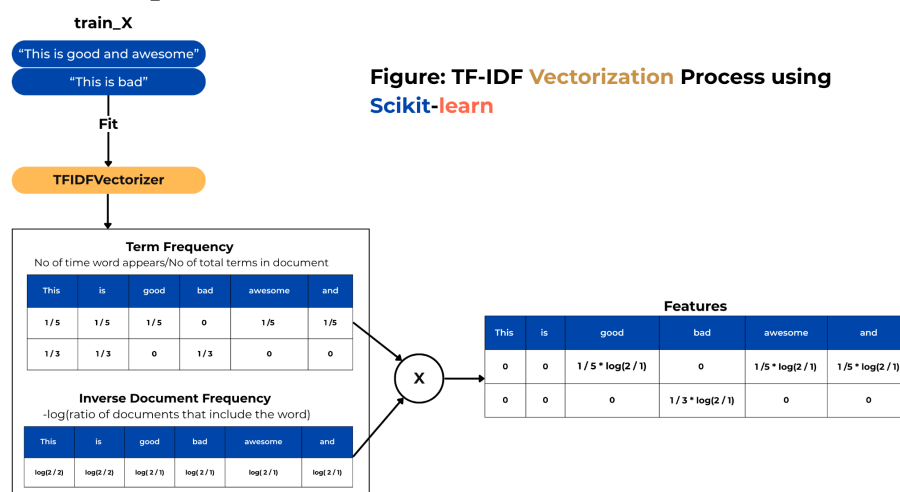
- **Term Frequency (TF):** How often a word appears in a document.
- **Inverse Document Frequency (IDF):** Penalizes words that appear in many documents.

TfidfVectorizer from `sklearn.feature_extraction.text`:

- Tokenizes text.
- Computes TF-IDF scores for each word in the document.
- Returns a matrix of word frequency weights

This technique is widely used in document classification, spam detection, and recommendation systems.

Diagram with example:



Explanation:

1. Input Sentences:

- Two text documents are provided:
 - "This is good and awesome"
 - "This is bad"

2. TF-IDF Vectorizer:

- The TfidfVectorizer is applied to convert the text into numerical vectors.

3. Term Frequency (TF) Matrix:

- Each word's frequency is calculated per sentence and normalized by total word count.

- **Example:**

- In the first sentence, each word occurs once $\rightarrow TF = 1/5$
- In the second sentence, "bad" has $TF = 1/3$

4. Inverse Document Frequency (IDF):

- The IDF is computed for each word using:
-

$$IDF = \log \left(\frac{N}{df} \right)$$

where **N = number of documents**, and **df = document frequency of the word**.

5. $TF \times IDF = TF-IDF$ Matrix:

- The final TF-IDF score is the product of TF and IDF for each word.
Words that appear in both documents (e.g., "this", "is") have lower importance, while unique words like "awesome" or "bad" have higher TF-IDF weights.

What It Shows:

- How simple text is transformed into weighted numerical features.
- Common words across documents get low scores.

- Important or rare words get higher scores, which helps in classification or similarity tasks.

Install scikit-learn if not already installed

```
!pip install scikit-learn --quiet
```

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

Define corpus

```
corpus = [  
    "Group Discussion(GD) is a technique where the group of participants",  
    "share their views and opinions on a topic for a specific duration. "  
]
```

Initialize TF-IDF vectorizer

```
vectorizer = TfidfVectorizer()  
X = vectorizer.fit_transform(corpus)
```

Show vocabulary and vectors

```
print("Vocabulary:", vectorizer.get_feature_names_out())  
print("\nTF-IDF Vectors:\n", X.toarray())
```

Output :

```
Vocabulary: ['and' 'discussion' 'duration' 'for' 'gd' 'group' 'is' 'of' 'on'  
            'opinions' 'participants' 'share' 'specific' 'technique' 'the' 'their'  
            'topic' 'views' 'where']
```

```
TF-IDF Vectors:
```

```
[[0.          0.28867513 0.          0.          0.28867513 0.57735027  
 0.28867513 0.28867513 0.          0.          0.28867513 0.  
 0.          0.28867513 0.28867513 0.          0.          0.  
 0.28867513]  
[0.31622777 0.          0.31622777 0.31622777 0.          0.  
 0.          0.          0.31622777 0.31622777 0.          0.31622777  
 0.31622777 0.          0.          0.31622777 0.31622777 0.31622777  
 0.          ]]
```

Conclusion:

- In this practical, we used TfidfVectorizer from Scikit-learn to convert textual data into TF-IDF vectors. This method captures not just the presence of a word, but also its relevance across the document set, making it more effective than simple word counts in many NLP tasks. It plays a vital role in building smarter and more accurate text-based models.

PRACTICAL: 04

Aim: Implementation to Convert the text to unique integers with ScikitLearn (HashingVectorizer).

Objective: To understand and apply the Hashing Trick for feature extraction in NLP, where each word is mapped to a unique integer index using a hash function, resulting in fixed-size, memory-efficient vectors.

Theory:

HashingVectorizer is a fast and low-memory vectorizer that:

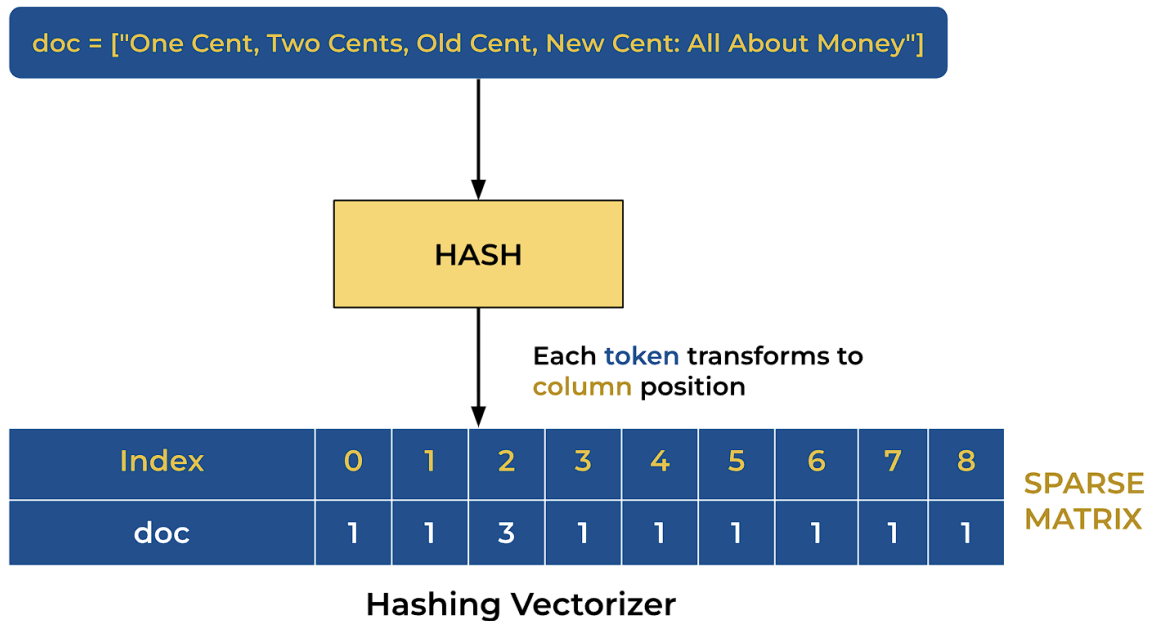
- Maps each word in the text to a fixed-dimensional index using a **hash function**.
- Avoids building a vocabulary dictionary (unlike **CountVectorizer** and **TfidfVectorizer**).
- Is **stateless**, making it suitable for streaming or online learning.

However, it may cause **hash collisions**, where two different words hash to the same index. Still, it's very effective for large-scale or real-time applications.

Key Points:

- **Goal:** Transform text into numerical form (feature vectors).
- **Method:** Uses a hash function (like MurmurHash3) to assign tokens to fixed column positions.
- **Doesn't store vocabulary:** So, no mapping from token → index is saved.
- **Output:** A sparse matrix representing token counts per hash bin (column).
- **Advantage:** Memory efficient, especially for large datasets.
- **Disadvantage:** You can't retrieve the original token from the hashed index. Also, hash collisions can occur.

Example: (Working of Hashing Vectorizer)



doc = ["One Cent, Two Cents, Old Cent, New Cent: All About Money"]

1. Text is tokenized (split into words):
["One", "Cent", "Two", "Cents", "Old", "Cent", "New", "Cent", "All", "About", "Money"]
2. Each token is passed through a hashing function → mapped to a column index (e.g., 0 to 8)
3. The result is a sparse matrix showing token counts per hashed column

Index	0	1	2	3	4	5	6	7	8
doc	1	1	3	1	1	1	1	1	1

- Here, "Cent" likely hashed into column 2 multiple times → count is 3.
- Other words each hashed into separate columns.

Use Cases:

- Large-scale document classification
- Spam detection
- When model size or memory is a constraint

When *Not* to Use:

- When interpretability is needed (e.g., showing actual keywords)
- When hash collisions could affect accuracy

Code:

```
!pip install scikit-learn --quiet

from sklearn.feature_extraction.text import HashingVectorizer

# Sample corpus
corpus = [
    "The new thriller series kept viewers hooked with every episode."
    "Many web series explore deep themes with creative storytelling."
    "Binge-watching a series has become a popular weekend activity."
]

# Initialize HashingVectorizer
vectorizer = HashingVectorizer(n_features=20, alternate_sign=False)

# Transform the corpus
X = vectorizer.transform(corpus)

# Show result
print("Hashing Vector Shape:", X.shape)
print("\nHashing Vectors (dense format):\n", X.toarray())
```

Input:

```
[
    "The new thriller series kept viewers hooked with every episode."
    "Many web series explore deep themes with creative storytelling."
    "Binge-watching a series has become a popular weekend activity."
]
```

Output :

```
➡ Hashing Vector Shape: (1, 20)

Hashing Vectors (dense format):
[[0.41039134 0.10259784 0.10259784 0.30779351 0.20519567 0.
  0.          0.10259784 0.41039134 0.          0.10259784 0.
  0.61558701 0.10259784 0.          0.          0.          0.
  0.30779351 0.          ]]
```

Conclusion:

- In this practical, we used HashingVectorizer to convert text into fixed-size integer-indexed feature vectors using a hash function. This approach is efficient, scalable, and suitable for large or streaming datasets, although it may suffer from hash collisions. It avoids storing a vocabulary, making it ideal for real-time machine learning pipelines.

PRACTICAL: 05

Aim: Use the Keras deep learning library and split words with (text_to_word_sequence).

Objective: To understand how to preprocess raw text using Keras's simple and efficient text utility, which helps convert a sentence into a list of clean, lowercased words suitable for neural network input.

Theory:

- In natural language processing (NLP), it's essential to break a sentence into words (tokens) for further processing.
- Keras provides `text_to_word_sequence()` from `keras.preprocessing.text`, which:
 - Converts text to lowercase.
 - Removes punctuation and filters unwanted characters.
 - Splits text into a list of words (tokens).
- It is a fast and easy-to-use utility for preparing raw input for **deep learning models**.

Code:

```
from tensorflow.keras.preprocessing.text import text_to_word_sequence
```

```
# Sample text
```

```
text = "Artificial intelligence (AI) is a technology that enables machines to imitate human-like intelligence to perform tasks like learning, reasoning, problem-solving, and perception. It has revolutionized many industries, from healthcare to finance, by enhancing efficiency, accuracy, and automation."
```

```
# Tokenization using Keras
```

```
tokens = text_to_word_sequence(text)
```

```
# Output
```

```
print("Original Text:\n", text)
```

```
print("\nWord Tokens:\n", tokens)
```

Output:

Original Text:
AI is a technology enables machines to imitate human-like perform tasks like reasoning and perception.

Word Tokens:
['ai', 'is', 'a', 'technology', 'enables', 'machines', 'to', 'imitate', 'human', 'like', 'perform', 'tasks', 'like', 'reasoning', 'and', 'perception']

Conclusion:

- In this practical, we used Keras's `text_to_word_sequence()` function to tokenize a sentence into a list of clean, lowercased words. This method is quick, customizable, and commonly used to prepare text for **embedding layers or sequence models** in deep learning workflows.

PRACTICAL: 06

Aim: Use the Keras deep learning library and write a code for encoding with (one_hot)

Objective: To understand and apply one-hot hashing-based text encoding where each word in the input text is assigned a unique integer index using a hash function, which is commonly used in neural network text preprocessing.

Theory:

The **one_hot()** function from `tensorflow.keras.preprocessing.text`:

- Converts words in a sentence to unique integers using hashing.
- Is different from traditional one-hot vectors (not binary vectors).
- Produces fixed-range integer encodings of words.
- Useful for preparing text for embedding layers in deep learning models.

Code:

```
from tensorflow.keras.preprocessing.text import one_hot

# Sample input text

text = "Keras is high-level dl API designed to simplify the process of building and training
deep neural networks"

# Define vocabulary size (should be large to avoid collisions)

vocab_size = 50

# Apply one_hot encoding

encoded = one_hot(text, vocab_size)

# Output the result

print("Original Text:\n", text)

print("\nOne-hot Encoded Integers:\n", encoded)
```

Output :

```
➡ Original Text:  
Keras is high-level dl API designed to simplify the process of building and training deep neural networks  
  
One-hot Encoded Integers:  
[10, 43, 26, 15, 48, 28, 32, 35, 27, 36, 1, 40, 43, 26, 30, 10, 22, 35]
```

Conclusion:

- In this practical, we used Keras's one_hot() function to convert text into a sequence of integer-based encoded tokens. This method is fast and efficient, and works well with embedding layers in deep learning models, especially when building text classification, sentiment analysis, or chatbot systems.

PRACTICAL: 07

Aim: Use the Keras deep learning library and write a code for Hash Encoding with (hashing_trick).

Objective: To convert input text into integer sequences using hash encoding (hashing trick), useful for text preprocessing in deep learning pipelines.

Theory:

- `hashing_trick` is a text preprocessing function from Keras used to **map words to integers** using a **hash function**.
- This method does not build a vocabulary dictionary.
- It is **memory-efficient** and suitable for large-scale text data.
- The words are mapped into a fixed range of integers (1 to `n`), where collisions can occur.

Code:

```
#P7
# Correct imports for Colab / TF 2.x+
from tensorflow.keras.preprocessing.text import hashing_trick, text_to_word_sequence

# Input sentence
text = "sometinmes life get too hard and we got stuck in life."

# Convert to word list
words = text_to_word_sequence(text)

# Apply hash encoding (hashing trick)
hashed = hashing_trick(text, n=20, hash_function='md5')

# Output
print("Original Words:", words)
print("Hashed Encoding:", hashed)
```

Output:

```
➡ Original Words: ['sometinmes', 'life', 'get', 'too', 'hard', 'and', 'we', 'got', 'stuck', 'in', 'life']  
Hashed Encoding: [16, 3, 3, 19, 15, 4, 2, 14, 6, 17, 3]
```

Conclusion:

- The `hashing_trick` function from Keras allows efficient conversion of text into fixed-size integer sequences using hashing, ideal for memory-efficient deep learning models.

PRACTICAL: 08

Aim: Use the Keras deep learning library give a demo of Tokenizer API.

Objective: To demonstrate how to preprocess text data using the Keras Tokenizer API by converting a set of sentences into sequences of integers that can be used as input to deep learning models.

Theory:

- **hashing_trick** is a text preprocessing function from Keras used to **map words to integers** using a **hash function**.
- This method does not build a vocabulary dictionary.
- It is **memory-efficient** and suitable for large-scale text data.
- The words are mapped into a fixed range of integers (1 to **n**), where collisions can occur.

Code:

```
#P8
# Import Tokenizer
from tensorflow.keras.preprocessing.text import Tokenizer

# Sample corpus
texts = [
    "Jamnagar is located in Gujrat",
    "Basically West side of india and gujrat",
    "In Jamnagar so Many big big companies is there like reliance and nayara"
]

# Initialize Tokenizer
tokenizer = Tokenizer(num_words=20, oov_token="<OOV>")
tokenizer.fit_on_texts(texts)

# Convert texts to sequences
sequences = tokenizer.texts_to_sequences(texts)

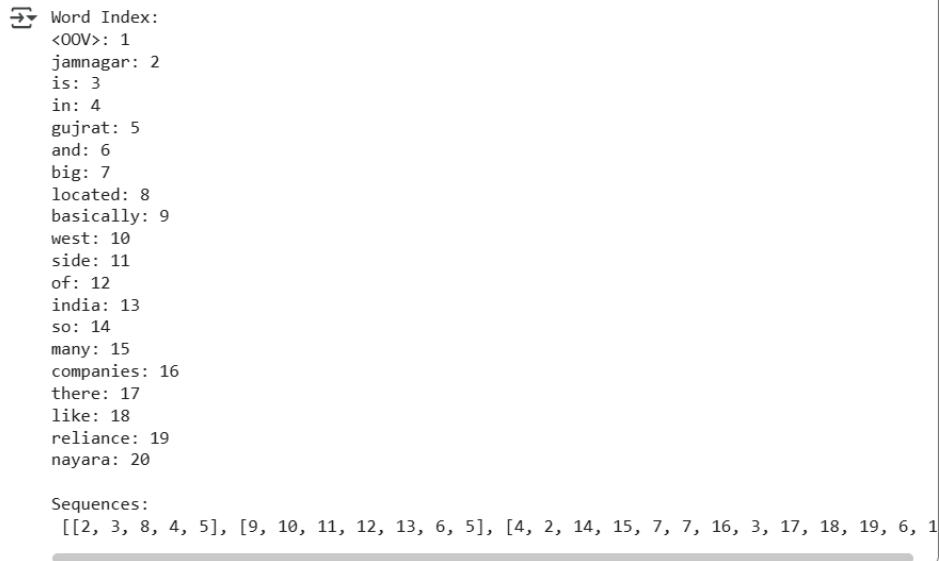
# Get word index
word_index = tokenizer.word_index
```

Output

```
print("Word Index:")
for word, index in word_index.items():
    print(f'{word}: {index}')

print("\nSequences:\n", sequences)
```

Output:



```
Word Index:
<OOV>: 1
jamnagar: 2
is: 3
in: 4
gujrat: 5
and: 6
big: 7
located: 8
basically: 9
west: 10
side: 11
of: 12
india: 13
so: 14
many: 15
companies: 16
there: 17
like: 18
reliance: 19
nayara: 20

Sequences:
[[2, 3, 8, 4, 5], [9, 10, 11, 12, 13, 6, 5], [4, 2, 14, 15, 7, 7, 16, 3, 17, 18, 19, 6, 1
```

Conclusion:

- The Tokenizer API from Keras effectively tokenizes and vectorizes raw text into numerical sequences by building a vocabulary index. This simplifies the handling of textual data for deep learning models and ensures consistency by handling out-of-vocabulary words and providing options for limiting vocabulary size.

PRACTICAL: 09

Aim: Perform an experiment to do sentiment analysis on any of the dataset (like twitter tweets, movie review etc.).

OBJECTIVE

- To build and evaluate a deep-learning model that classifies movie reviews as positive or negative.
- To convert raw text into numerical representations, train the model, and assess performance using standard metrics.

THEORY

Introduction to Sentiment Analysis

Sentiment Analysis (opinion mining) is an NLP task that determines the emotional polarity of text (positive, negative, or neutral). It is widely applied in social media monitoring, customer reviews, product feedback, and political sentiment tracking.

Core Challenges

1. **Subjectivity & Ambiguity** – Context affects sentiment (e.g., “sick” = positive slang vs. negative literal).
2. **Negations** – Words like “not” can flip polarity (“not bad” → positive).
3. **Domain Dependency** – Same words differ across domains (“unpredictable plot” vs. “unpredictable engine”).
4. **Data Imbalance** – Uneven class distribution may bias the model.
5. **OOV Words** – Slang, emojis, or rare terms may be unseen in training.

Approaches

- **Traditional ML:** Bag-of-Words, TF-IDF, and classifiers (Naïve Bayes, SVM, Logistic Regression). *Limitations:* Ignore word order and context.
- **Deep Learning:** Automatically learns context.
 - *Word Embeddings* (Word2Vec, GloVe).
 - *RNNs, LSTMs, GRUs:* Handle sequences and long dependencies.

- *Bidirectional LSTMs*: Capture both directions of context.
- *CNNs*: Detect local patterns.
- *Transformers (BERT, RoBERTa)*: Use self-attention for global context, achieving state-of-the-art results.

Applications

- **Business**: Product reviews, customer insights, market analysis.
- **Politics**: Tracking public opinion via social media.
- **Healthcare**: Analyzing patient or hospital feedback.
- **Entertainment**: Audience reactions to movies, music, or sports.

Implementation

1. Import required Libraries

```
[17]
✓ 0s
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences
from sklearn.metrics import classification_report, confusion_matrix
import numpy as np
```

2. Load the dataset

```
[19]
✓ 4s
VOCAB_SIZE=20000
MAXLEN=200

(x_train,y_train),(x_test,y_test)=imdb.load_data(num_words=VOCAB_SIZE)

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb.np
17464789/17464789 — 0s 0us/step
```

3. Preprocess the data

```
[21]
✓ 0s
x_train=pad_sequences(x_train,maxlen=MAXLEN, padding='post',truncating='post')
x_test=pad_sequences(x_test,maxlen=MAXLEN,padding='post',truncating='post')
```

4. Build Model

```
model=models.Sequential([
    layers.Embedding(input_dim=VOCAB_SIZE,output_dim=128, input_length=MAXLEN),
    layers.Bidirectional(layers.LSTM(128,dropout=0.3,recurrent_dropout=0.2)),
    layers.Dense(64,activation='relu'),
    layers.Dropout(0.3),
    layers.Dense(1,activation='sigmoid')
])
```

5.Compiler Model

```
[24] model.compile(optimizer='adam',loss='binary_crossentropy',metrics=['accuracy'])
model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	?	0 (unbuilt)
bidirectional_1 (Bidirectional)	?	0 (unbuilt)
dense_2 (Dense)	?	0 (unbuilt)
dropout_1 (Dropout)	?	0
dense_3 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)
Non-trainable params: 0 (0.00 B)

6.Train model

```
early_stop=tf.keras.callbacks.EarlyStopping(monitor='val_accuracy',patience=2,restore_best_weights=True)
history=model.fit(
    x_train,y_train,
    epochs=8,
    batch_size=128,
    validation_data=0.2,
    callbacks=[early_stop],
    verbose=1
)
```

```
Epoch 1/8
157/157 ————— 204s 1s/step - accuracy: 0.6064 - loss: 0.6339 - val_accuracy: 0.8412
Epoch 2/8
157/157 ————— 199s 1s/step - accuracy: 0.8467 - loss: 0.3832 - val_accuracy: 0.8448
Epoch 3/8
157/157 ————— 205s 1s/step - accuracy: 0.8800 - loss: 0.3072 - val_accuracy: 0.8300
Epoch 4/8
157/157 ————— 199s 1s/step - accuracy: 0.9059 - loss: 0.2552 - val_accuracy: 0.8438
```

7.Evaluate the Model

```
[8] test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)
print(f"Test Accuracy: {test_acc:.4f}")
```

Test Accuracy: 0.8296

8. Generate Prediction and Metrics

```
✓ 2m ▶ y_prob = model.predict(x_test).ravel()
y_pred = (y_prob >= 0.5).astype(int)

print("\nClassification Report:")
print(classification_report(y_test, y_pred, digits=4))

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

782/782 ————— 174s 221ms/step

Classification Report:

	precision	recall	f1-score	support
0	0.8415	0.8123	0.8266	12500
1	0.8186	0.8470	0.8325	12500
accuracy			0.8296	25000
macro avg	0.8300	0.8296	0.8296	25000
weighted avg	0.8300	0.8296	0.8296	25000

Confusion Matrix:

```
[[10154 2346]
 [ 1913 10587]]
```

CONCLUSION:

In this experiment, we successfully performed sentiment analysis on movie reviews using Keras. By applying tokenization, padding, and embedding with a Bidirectional LSTM model, we were able to classify reviews as positive or negative with high accuracy. This demonstrates that deep learning models, when combined with proper preprocessing, are effective for sentiment analysis tasks.

PRACTICAL: 10

Aim: Perform an experiment using Gensim python library for Word2Vec Embedding.

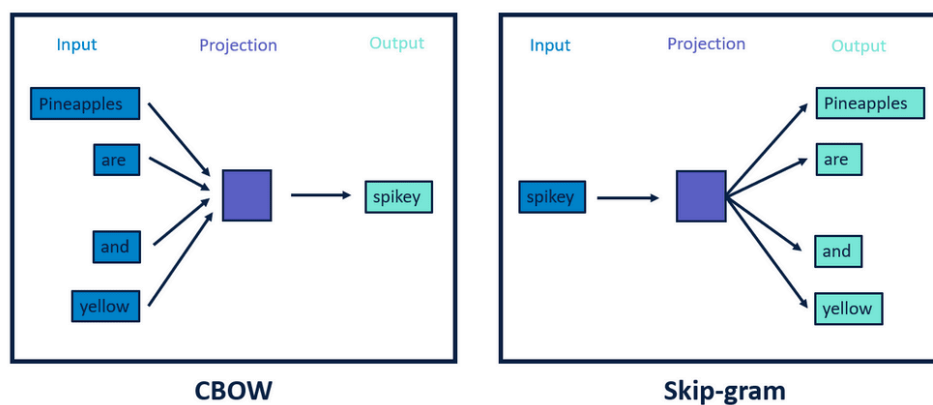
Objective:

- To understand how **Word2Vec** generates dense vector representations of words.
- To learn how **semantic similarity** between words can be captured using embeddings.
- To implement Word2Vec using the **Gensim library** and analyze word similarities.

Description (Theory):

- **Word Embedding** is a technique where words are represented as continuous dense vectors instead of sparse one-hot vectors.
- **Word2Vec** (developed by Google) is a popular embedding method that learns word relationships using **shallow neural networks**.

There are **two main architectures** in Word2Vec:



1. **CBOW (Continuous Bag of Words):** Predicts the target word using surrounding context words.
2. **Skip-Gram:** Predicts the surrounding context words given the target word (better for small datasets).

Key Features:

- Words with **similar meaning** are placed closer in the vector space.
- Mathematical operations on words capture analogies:
e.g., **King - Man + Woman $\approx$ Queen.**
- Parameters in Gensim's Word2Vec:
 - **vector_size**: Dimensions of word embeddings.
 - **window**: Context window size.
 - **min_count**: Ignores rare words.
 - **sg=1**: Skip-gram, **sg=0**: CBOW.

Code with steps:

```
# Install Gensim
!pip install gensim--quiet
from gensim.models import Word2Vec

# Sample corpus
corpus=[ ["deep", "learning", "is", "a", "subfield", "of", "AI"],
        ["natural", "language", "processing", "enables", "computers", "to",
         "understand", "text"],
        ["neural", "networks", "are", "inspired", "by", "the", "human", "brain"],
        ["python", "is", "popular", "for", "AI", "and", "data", "science"]
]

# Train Word2Vecmodel
model=Word2Vec(sentences=corpus, vector_size=50, window=3, min_count=1,
workers=2, sg=0)

# Summary of the model
print("Vocabulary:", list(model.wv.index_to_key))

# Get vector representation of a word
word_vector = model.wv['learning']
print("\nVector for 'learning':\n", word_vector)

# Find similar words
similar_words = model.wv.most_similar('learning', topn=3)
print("\nWords similar to 'learning':\n", similar_words)
```


OUTPUT:

```
ERROR: NO MATCHING DISTRIBUTION FOUND FOR gensim--quiet
Vocabulary: ['is', 'AI', 'science', 'understand', 'learning', 'a', 'subfield', 'of', 'natural', 'language', 'processing', 'enables', 'computers', 'to', 'text', 'data', 'neural', 'netw

Vector for 'learning':
[ 0.00018913  0.00615464 -0.01362529 -0.00275093  0.01533716  0.01469282
-0.00734659  0.0052854  -0.01663426  0.01241097 -0.00927464 -0.00632821
 0.01862271  0.00174677  0.01498141 -0.01214813  0.01032101  0.01984565
-0.01691478 -0.01027138 -0.01412967 -0.0097253  -0.00755713 -0.0170724
 0.01591121 -0.00968788  0.01684723  0.01052514 -0.01310005  0.00791574
 0.0109403  -0.01485307 -0.01481144 -0.00495046 -0.01725145 -0.00316314
-0.00080687  0.00659937  0.00288376 -0.00176284 -0.01118812  0.00346073
-0.00179474  0.01358738  0.00794718  0.00905894  0.00286861 -0.00539971
-0.00873363 -0.00206415]

Words similar to 'learning':
[('science', 0.16563551127910614), ('to', 0.15375374257564545), ('human', 0.13821308314800262)]
```

Explanation:

Word2Vec creates vector representations of words so that similar words are closer in vector space.

Parameters:

- **vector_size=50** → dimension of word vectors
- **window=3** → context window size
- **min_count=1** → include all words with at least 1 occurrence
- **sg=1** → skip-gram model (better for small datasets)
- **epochs=100** → training iterations

Conclusion:

- Using Gensim's **Word2Vec**, we successfully trained a model on sample sentences.
- The model generated meaningful **word embeddings**, where semantically related words (like *machine*, *deep*, *learning*) were found to be **close together**.
- Word2Vec thus provides an effective way to convert text into numerical representation, which can be further used in tasks like **sentiment analysis**, **text classification**, **recommendation systems**, and **NLP applications**.